8 - Linked Lists

A linked list is built on relative position much like standing in a line allows us to build a linked structure. Consider that you can successfully stand in line without ever knowing or managing your absolute position in any way. Instead you maintain your position in line simply by keeping track of who you are adjacent to. This is the underlying idea of relative position, *i.e.* maintaining position with respect to references to neighboring elements.

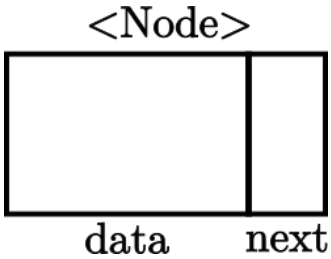
Singly Linked List

The term "singly" in singly linked list describes the number of references used to maintain relative position. Based on the "standing in line" analogy, a singly linked version of this line would be each person only needs to keep track of who is standing behind them in order to maintain the integrity of the line.

The key to this structure is that each element must keep track of neighboring elements and we will need a structure that supports this reference based structural information.

List Node

The list node is a very simple container that will encapsulate together a data element and in the most simple form a reference to the next element. Consider a line where you keep track of who is behind you to organize the list. You are a node and you are also tracking a reference to the “next” following element behind you.



Since Java is a strongly typed language, both of these elements must have associated types declared. The following `class` is one possible way to express the above Node type.

```
class Node {
    int data;
    Node next;
}
```

In the above example, we are assigning the `data` field an `int` type. This is primarily for illustrative purposes because it would be more practical to define a single Node type capable of holding a wider range of types of data. To do so, we could make the type of the data field a reference to a generic Jav `Object`, but to use this effectively, we will need a better understanding of polymorphism in Java.

```
class Node {
    Object data;
    Node next;
}
```

The real oddity for novice programmers is seeing an class contain a reference to an object of the same type. It is very normal for this type of referencing to occur. The disjoint in reasoning comes because a student is often still thinking of reference types as objects and not as reference types. Back in our standing in line analogy, the node serves as us "tracking the next person" in line not as a "person within a person". In other words, it is not some recursive situation, *i.e.* a node object does not contain another node, but instead involves a reference, *i.e.* a node points to another node.

Nodes can of course be more complex and we will see other data structures with similar nodes but more properties later in discussion.

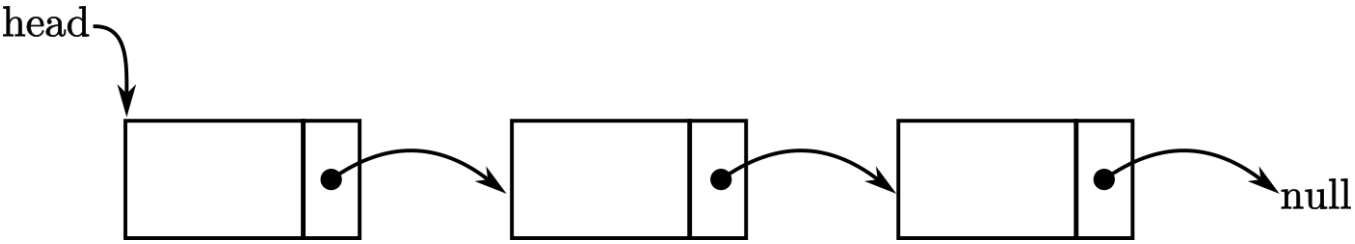
As mentioned in the introduction to generic functions in the chapter 5 discussion, a data structure can be defined with generic types as well. The details of this will be covered more in the chapter 8, but the `Node` structure could instead be defined as the following to allow for an explicit type for the `data` element at compile time while still supporting any type:

```
class Node <T> {
    T data;
    Node<T> next;
}
```

The rest of this chapter will first focus on defining a linked list using an `int` primitive type.

Minimal Singly Linked List

Any linked list will be built out of nodes and at the very least a head pointer. The **head pointer** refers to the first element in the list and through the next pointer of the first element, *i.e.* pointed to by the head, subsequent elements are linked and accessible. The end of the list is indicated by the last node-in-the-list's next pointing to null which indicates that nothing follows it.



Linked lists can have more complex construction, but the above design is remarkably capable of supporting all important list operations although some operations can be made more efficient by adding additional complexity to the linked list structure. We will slowly add additional features here, but it is important to understand that you only need to implement more features for wider ranges of linked list applications and in some systems, *e.g.* operating systems, you may encounter linked lists that are as simple as possible to minimize their space complexity.

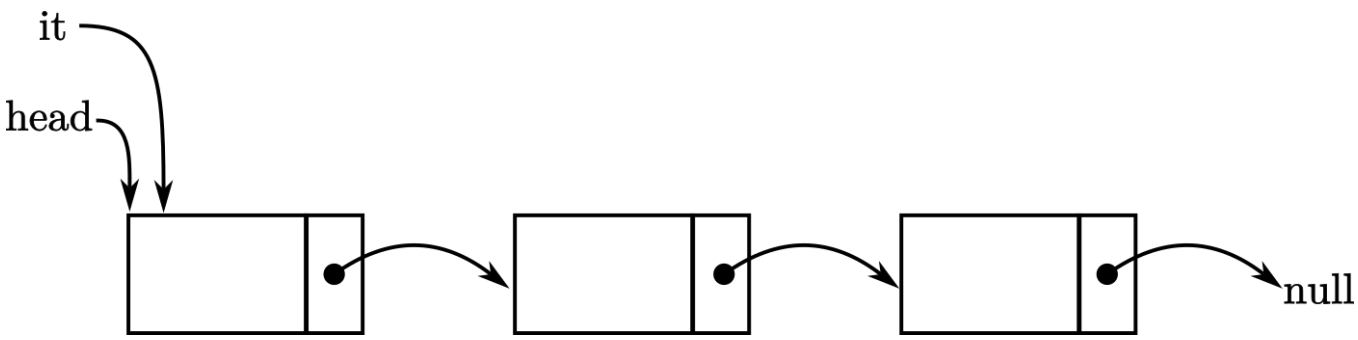
Iterating Through a Linked List

> **Iterators**
>
> An **iterator** is a pointer with the specific role to move through the data. It is a temporary pointer used to point to existing data so generally it will not be assigned to a new object. Instead it is assigned to point to an existing object, so it is a shallow copy of an existing reference. For non-linear, complex data structures, indexing stops making sense and it is necessary to find an alternative means to move through a data structure rather than using a simple, indexed `for` loop. The iterator is an extremely flexible tool that facilitates traversal of any data structure and it is very important to learn what an iterator is, how it works, and how to use it. The linked list is the simplest structure to apply and learn this tool, so it is critical to adopt it now. As a result, we will generally abandon the `for` loop construct for linked lists as it is primarily focused on indexed based approaches and we will instead adopt the `while` loop construct to iterate over linked lists. If you continue to use the `for` loop here, it can lead to suboptimal, inefficient implementations, so you need to get comfortable with alternative iterative approaches that do not rely on conventional `for` loops.

We will use the variable name `it` when declaring an iterator. The label `it` is simple, easily recognizable, and distinct which evokes the idea of both iterator and the standard `i` used in indexing. A longer label, `iter`, may also be common in some organizations and conventions.

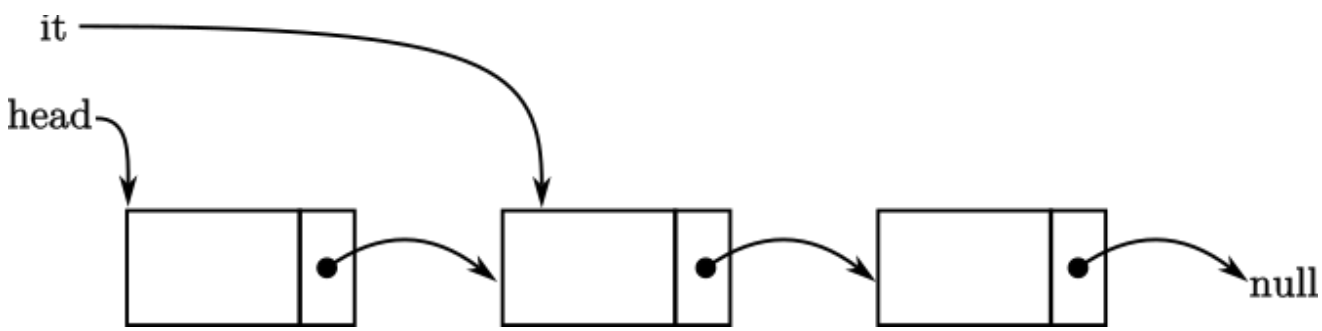
We can assign our iterator to the head to point to the first element. Remember that we are using this specifically to look at existing elements, so we are rarely assigning the iterator to a new node instance.

```
ListNode it = head;     // points to existing head, not a new node
```



To advance through the list, we can assign the iterator to its own next pointer.

```
ListNode it = head;
it = it.next;
```



This is where relative positioning takes over for a linked list. We can access any element relative to the head pointer.

```
head                    // first element
head.next               // second element
head.next.next          // third element
head.next.next...next   // ith element
```

But of course this is clunky and hard to effectively use which is where the more generalized approach of the iterator comes in:

```
ListNode it = head;     // first element
it = it.next;           // second element
it = it.next;           // third element
...                     // repeat the pattern
it = it.next;           // ith element
```

This generalized model is the basis of our iterative approach. In order to traverse the linked-list, we will move the iterator through the list until we reach the end of it by using the `it = it.next` statement. For example, we can traverse the entirety of the list with this simple `while` loop:

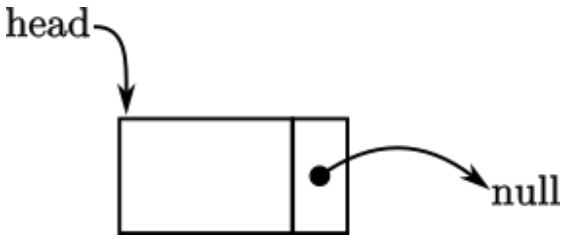
```
it = head;
while(it.next != null) {
    it = it.next;
}
```

There is a slight problem with the above `while` loop as it is difficult to do anything meaningful once the iterator goes beyond the tail of the list by reaching `null` but the solution is rather simple and only requires a slight adjustment to the condition the loop is based on.

Adding at the head

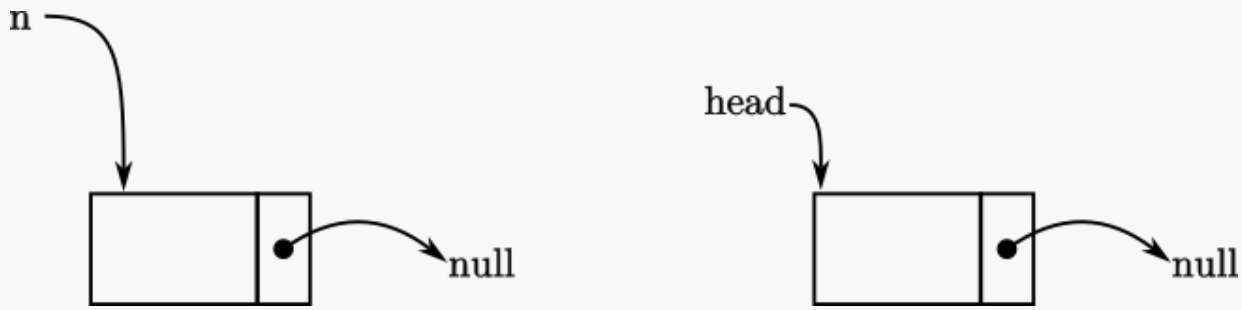
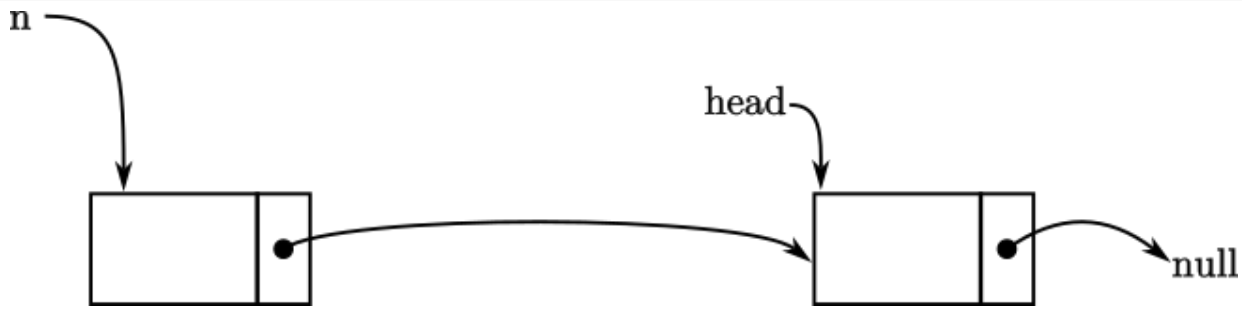
Recall the first operation we need to make a list usable is the ability to add to the linked list. Let's start with the assumption that we generally add at the head of the linked list and that this linked list is empty. To add the first element, we would use the following.

```
head = new Node(data);
```



We can add further elements at the head with the following

```
Node n = new Node(data);
n.next = head;
head = n;
```

1	
2	
3	

Now consider the efficiency of inserting a element into the head and contrast this efficiency with inserting an element into the first position of an array,

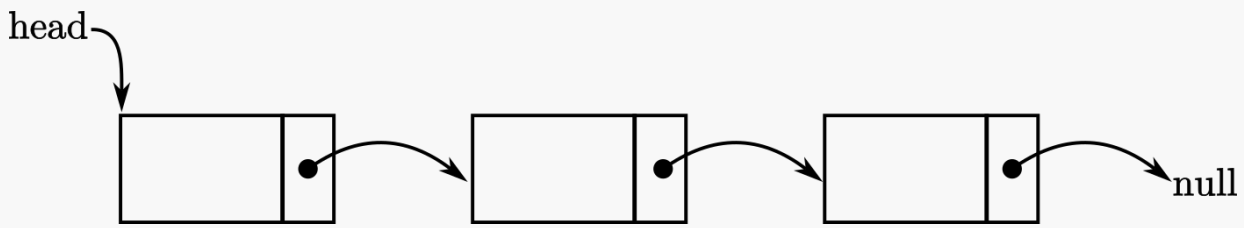
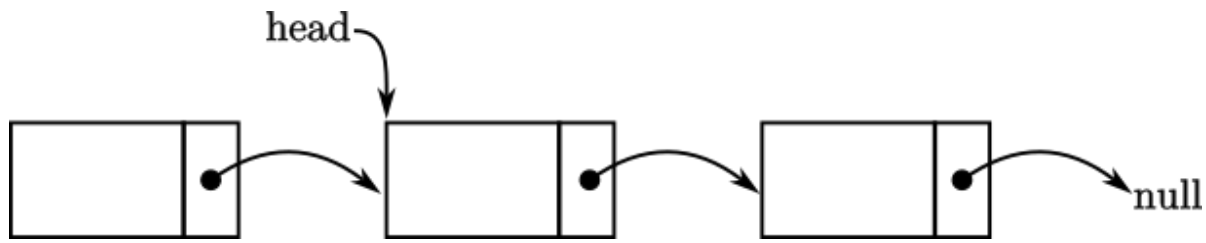
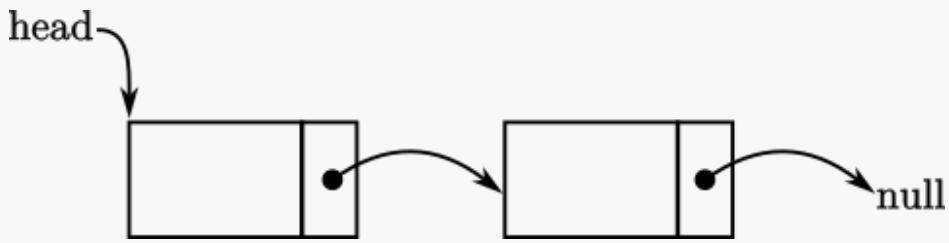
Given a non-empty linked list, regardless of how long the linked list is, it takes only three statements to insert at the head location; therefore, the relationship between n where n is the size of the linked list and the number of operations for insert at the head is always constant time complexity. Given a non-empty array, the relationship between n where n is the size of the array and the number of operations for insert into the first position of the array is always linear time complexity because to insert at the first index we must always first shift every element to the right to make room in the first position of the array regardless of whether or not the array is full. So if faced with an application that requires high frequency insertion at the beginning of a list, the most efficient choice is the linked list over an array.

Removing at the head

Now that we have demonstrated that we can add at the head of a linked list, can we also remove at the head of the linked list. And in particular, since we have linear time complexity with insertion at the head, do we have the same performance with removal at the head.

First we need to ask what is necessary in the implementation to remove the head and it turns out that it is a rather simple single line of code.

```
head = head.next;
```

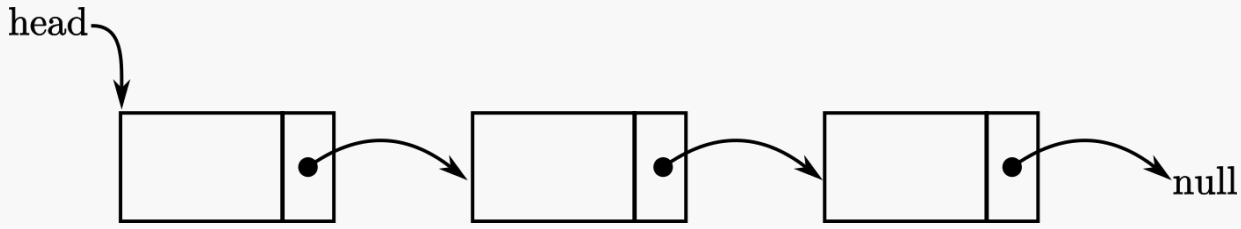
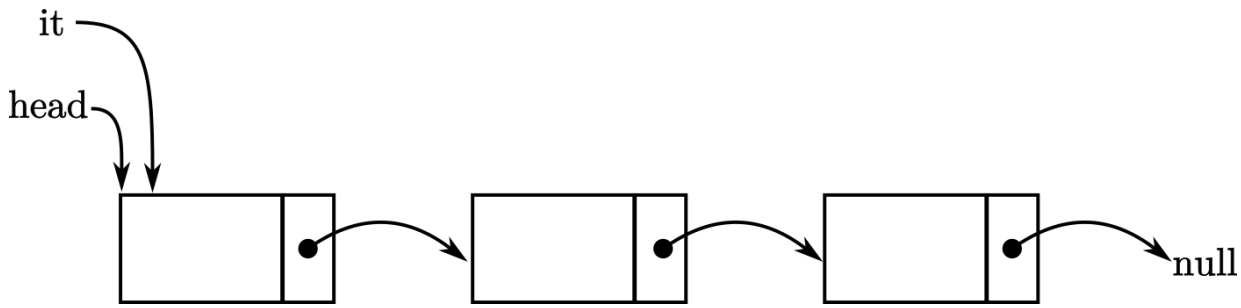
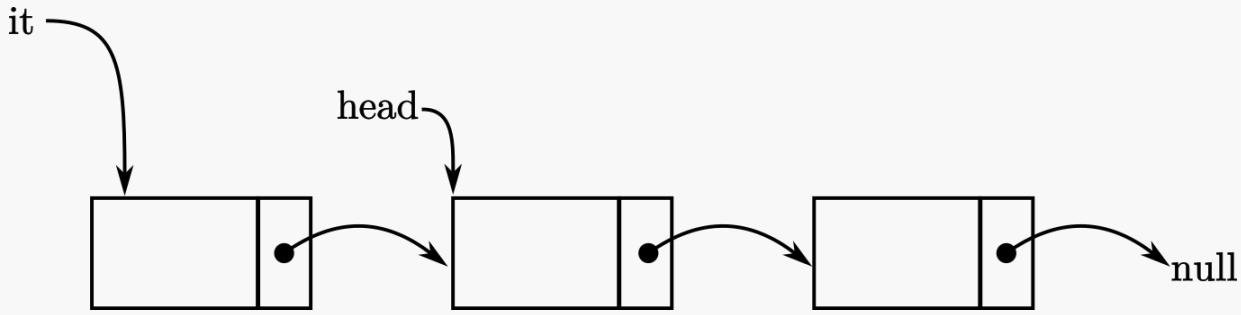
0	
1	
2	

In the step 1 of the code, the head is advanced to the second element via `head.next` which means that there is no longer a reference to the first element in the list. Since there are no references to the first element, the Java garbage collector comes along and cleans up the now orphaned first element and what was the second element is now the head of the list.

> **Garbage Collection**
>
> Garbage collection is a feature of some languages but not all. In C based languages there is no garbage collector and so this automatic cleanup of orphaned objects does not exist. This means it is the developer's responsibility to clean up memory manually in some languages and the linked list remove implementation in other languages may involve one or two more steps. The concepts proposed here are universal to all languages, but you will need to study the language features of your development language of choice and adjust an implementation accordingly. This is one of the reasons we use pseudo-code as a proto development language since it abstracts away language specific requirements.

If we needed to return the data in the first element in addition to removing it we can make a simple adjustment by adding an additional variable to retain the reference to the first element and return the data.

```
ListNode it = head;
head = head.next;
return it.data;
```

0	
1	
2	

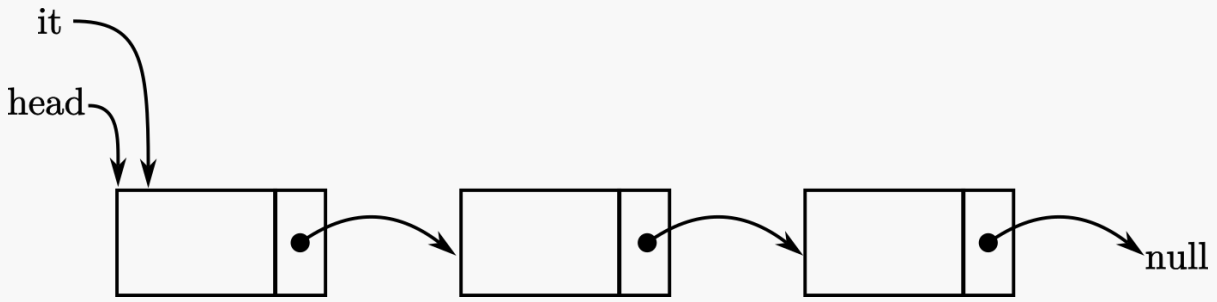
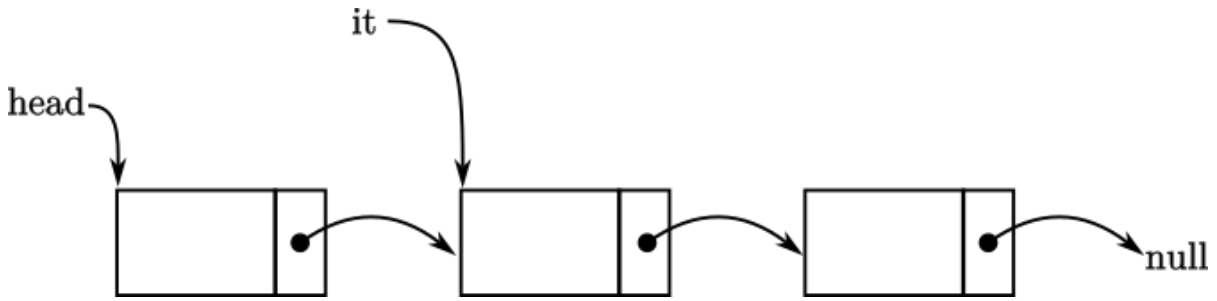
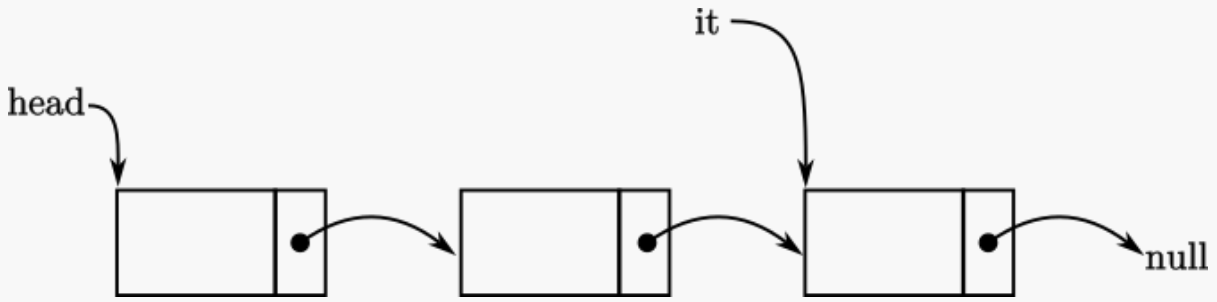
Again consider the efficiency instead with regard to removing an element at the head and contrast this efficiency with removing an element from the first position of an array,

Given a non-empty linked list, regardless of how long the linked list is, it takes a constant number of statements to remove from the head location; therefore, the relationship between n where n is the size of the linked list and the number of operations for removal from the head is constant time complexity. Given a non-empty array, the relationship between n where n is the size of the array and the number of operations for removal from the first position of the array is always linear time complexity because to remove from the first index we must always shift every element to the left to keep array elements aligned to the first position of the array. So if faced with an application that requires high frequency removal from the beginning of a list, the most efficient choice is the linked list over an array.

Removing at the tail

Suppose we need to remove an element at the tail of the linked list, how would we go about this? We only have a head pointer in the structure we have so far designed, so in order to remove from the tail, we need to first find the tail, so what does it look like to find the tail and what is the efficiency?

Finding the tail will require that we iterate through the linked list until we find the last element.

1	
ith	
nth	

Our search condition is whether or not `it.next` is null. We use `it.next` as a part of the conditional logic because we are searching for the last element. If `it` is equal to `null` then we run past the end of the list and our iterator is now invalid.

Remember, when iterating over a linked list use a `while` loop construct.

Ah, but this poses a problem. The iterator has gone past the node element that contains the pointer we need to fix. Instead of stopping at the last node, we really needed to stop at the second to last node and set that node's `next` pointer equal to `null`. While this is possible, it seems like we will be creating some unnecessarily complex code since we can always remove at the head in constant time. What if we can add at the tail and remove from the head, that will give us all the useful combinations of operations: add and remove at the same end, add and remove at different ends.

Adding at the tail

We are presented with the same problem as removal at the tail of the linked list, first we have to find the tail. Once the tail of the list has been found, what does adding to the tail look like?

```
it.next = new Node(data);
```

Note that finding the tail is the expensive part of adding at the tail and the code for actually adding at the tail once the tail is found is trivial. Again, consider the efficiency of inserting a element at the tail of the linked list assuming we have to find the tail and contrast this efficiency with inserting an element into the last position of an array.

Given a non-empty linked list, it takes n operations to insert at the tail location where n is the size of the linked list because we must iterate over the entire list in order to find the tail. Insertion at the tail is constant time because it just requires allocation of a new node and assignment of a single pointer to the new node.

Assuming the array is full, we will have to allocate a new array with sufficient space to copy the original array into and to append the element to insert, then we will need to perform the copy and insertion. Copying the array dominates this process as it requires n operations where n is the size of the array while allocation and insertion are at worst constant time, therefore the overall process to insert into a full array is linear time complexity.

The time complexity of both is rather dismal, but consider what would happen if we added a tail pointer to the linked list and maintained the tail pointer over all operations.

Inserting or indexing to any internal node

Inserting at the head, removing at the head, and adding at the tail are all constant time complexity operations for a singly linked-list with a head and a tail pointer, but consider the efficiency of other, more array inspired operations such as any indexed based operations like `get(int i)` or `add(element e, int i)`. The array was efficient at these types of operations because its fundamental design supports constant time access by index to any element while the linked list does not have this capability. Therefore, using a linked-list like an array is generally a poor choice because its efficiency in these operations will always be worse than what is possible using an array. This does not mean that we should not support indexed based operations in a linked-list; however, we should be sparing in using this operations, especially avoiding writing an indexed based `for` loop around a linked-list as this becomes $O(n^2)$ for a single `for` loop, and if we need indexed based operations, we can always copy the list data into an array at the cost of a constant time copy.

Why is an indexed based `for` loop around a linked-list for a single `for` loop $O(n^2)$ time complexity? Consider the following example:

```
LinkedList list = new LinkedList();
...                                         // append a lot of data to the list
for( int i=0; i < list.length(); i++ ) {
    Object o = list.get(i);                 // there is considerable hidden cost here
    System.out.println( o );
}
```

Why is there considerable hidden cost on the line involving the `get(int i )` call, because, backing this call for a linked list would be a linear search performed with each iteration of the above `for` loop. The above code combined with the implementation hidden behind the method call would actually look like the following:

```
LinkedList list = new LinkedList();
...                                         // append a lot of data to the list
for( int i=0; i < list.length(); i++ ) {
    Node it = list.head;
    for( int j=1; j < i; j++ ) {
        it = it.next;
    }
    Object o = it.data;
    System.out.println( o );
}
```

As you can see, the linked list `get(int i)` implementation requires a loop to back the operations. Using a `get` call in this case results in a nested loop with both loops bounded by n in the worst case; therefore, this is an $O(n^2)$ implementation even though we may have written this according the to first example above. The inner loop's contribution to the overall time complexity is easy to overlook and this hidden cost can make the algorithm inefficient. If we needed to use indexed based access, we should have chosen an array based data structure to begin with.

Note that the inner loop is designed as a `for` loop for this example because it maps well to indexed based problems, but most loop approaches inside the linked-list implementation would generally use `while` loops instead as suggested earlier.

Regardless of whether it is efficient or not, we can insert into the middle of the list by finding the node preceding the position at which we will perform the insertion or removal. Assuming the iterator references the node in front of the position to insert:

```
Node n = new Node(data);
// assumes that it references the element before the positon to insert
n.next = it.next;
it.next = n;
```

Note that the order in which the pointers are updated is important. Consider this alternative:

```
Node n = new Node(data);
// assumes that it references the element before the positon to insert
it.next = n;
n.next = it.next;
```

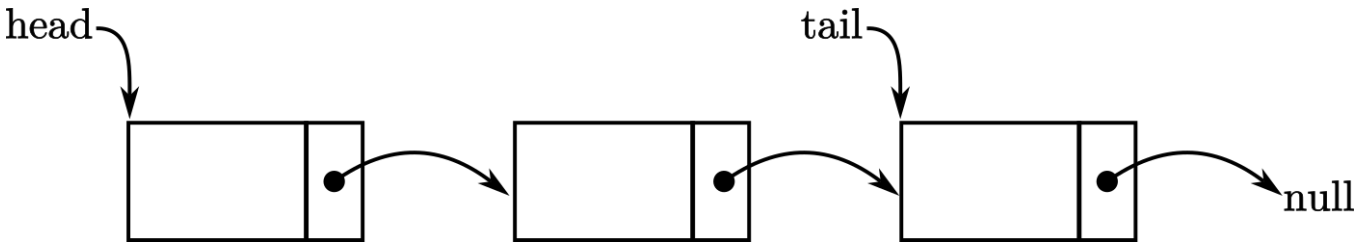
In the alternative, the iterator's next pointer is overwritten by the `it.next = n` instruction; however, the following instruction, `n.next = it.next`, operates on `it.next`. From the original example, we see that we first assign the new node's next element to the current element's (the iterator) next element before overwriting the iterator's next element in the second step. By reversing these two instructions in the second example, `it.next` no longer points to the current element's original next element in the list after the first step, so the second step results in a cycle where `n.next` points back to `n` which will result in an infinite loop in the list because there will no longer be a tail element pointing to null.

We can still provide methods for a linked list that index to an internal node, whether the method is add, remove, or get, but it is important to recognize that regularly using any of these methods is dramatically and negatively affecting the data structure and the program's performance. If the program must use indexed based approaches, the correct and optimal answer is to use an array because the array will perform in constant time for get and linear time for insert or remove for any element in the middle of the list. However, if we are performing these same operations only at the end of the list, we should use a linked list because we can support all of these operations in constant time if the operations are exclusive to the end of the list.

Wait above, we showed that we have to first find the tail element which takes linear time before performing the constant time insertion at the tail, so how can we generally support constant time access at the tail? The answer is to eliminate the search by maintaining a tail pointer at all times in the singly linked-list.

Singly Linked List with a Tail Pointer

Adding a tail pointer solves the problem of finding the tail when we need to access the tail end of the list. It adds some additional overhead to each operation because we will need to check a variety of conditions to know whether or not a list is empty which may trigger operations specific to the head, operations specific to the tail, or even operations involving both pointers such as when the linked list contains one element, *i.e.* the head and tail refer to the same element, but all of this cost is constant time.

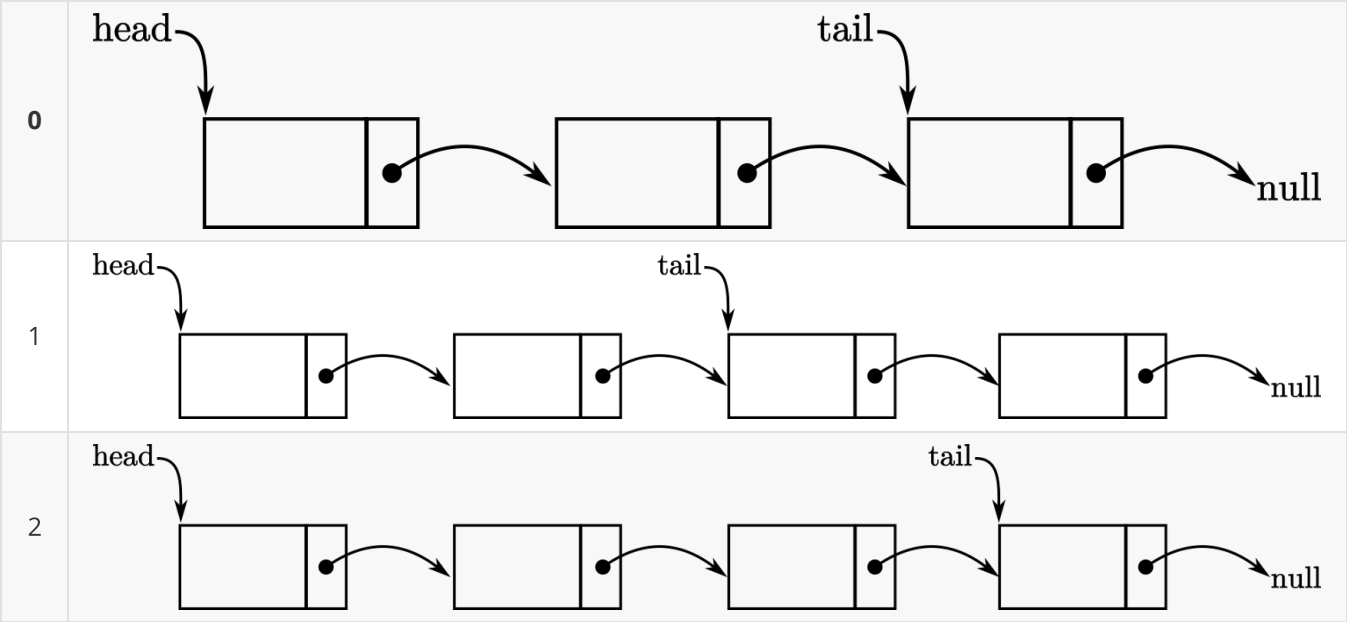


Adding the tail pointer does not affect adding and removal operations at the head, so we can still perform either operation at the head in constant time. It also does not resolve removal at the tail for a singly linked list as we would still need to find the second to last element in a costly iteration; however, it does solve the find problem for adding at the tail and reduces it from linear time to constant time insertion at the tail because the costly linear search is no longer required.

Adding at the tail

Recall adding at the tail once the tail was found was simply one line of code. Given a tail pointer, this code generally becomes:

```
tail.next = new Node(data);
tail = tail.next;
```



This resolves our problem of needing to support the different combinations when interacting at the ends. When adding at one end and removing at the other, we add at the tail and remove from the head. When adding at one end and removing at the same end, we add and remove at the head. This gives us constant time complexity for all the different combinations of end interaction and we will be able to support data structures that may rely on characteristics where end interaction is high frequency.

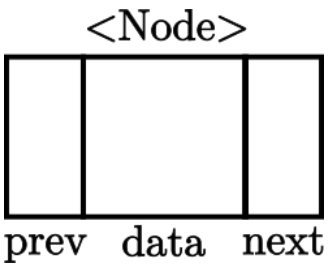
Doubly Linked List

We can continue to increase the complexity of the linked list structure. One change is to introduce a second link in every node such that each node can link to the node in front of it and to the node that follows it. In other words, the term "doubly" in doubly linked list describes that two references are used to maintain relative position in the list, and in the "standing in line" analogy, a doubly linked version of the line would be each person keeps track of both who is standing behind them and who is standing in from of them to maintain the integrity of the line.

While the doubly linked list is not necessary for interactions limited to either ends of the linked list, it does make it easier but not necessarily more efficient to remove elements in the middle of the linked list. However, it does make bidirectional traversal much easier, *i.e.* the ability to go back, which can be a necessary feature for specific problems.

Double Linked List Node (DLNode)

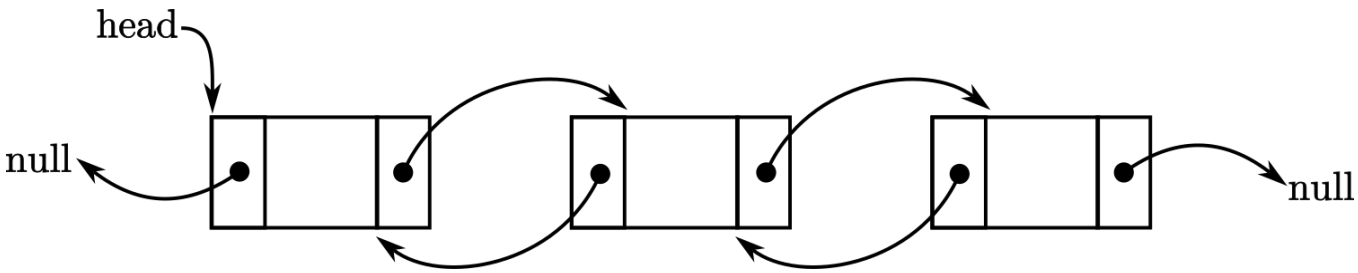
The doubly linked list node introduces a previous pointer or prev pointer to the node data structure. This previous pointer is linked to the node that precedes the node so that it is possible to move either forwards or backwards through the linked list.



This can be useful in very specific circumstances, but the increase in complexity can lead to additional bugs, so not all linked lists need to be doubly linked in order to function.

Minimal Doubly Linked List

Just as the minimal single linked list has only a head pointer, the minimal doubly linked list likewise only needs a head.



The convention to determine if we have reached the tail "end" of a list, *i.e.* a node with a next of null, is also used to determine if we have reached the head "end" of the list except it involves the prev pointer.

Inserting or removing a node

The doubly linked list facilitates insertion or removal of a new node in the middle of the list. The operation is still linear to locate the node; however, the implementation is fairly simple once we have identified the location to insert or remove.

For insertion, we need to find either the preceding or following node. Finding the preceding node is generally more standard. Assuming the iterator `it` points to the node in front of the position to insert, then inserting a node into the middle of the list follows these operations:

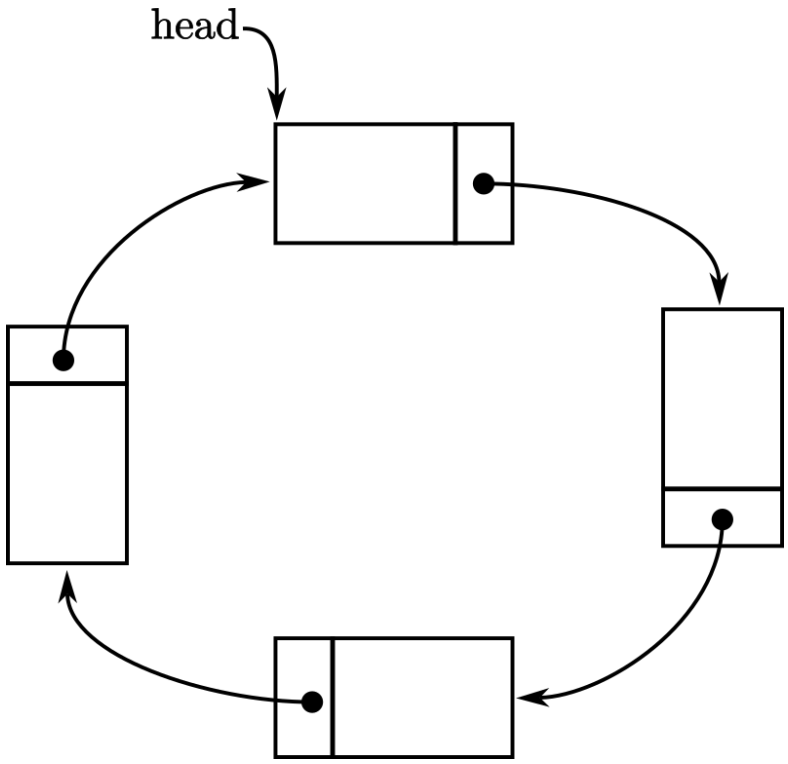
```
DLNode n = new DLNode(data);
// assuming it references the location before the position to insert
n.next = it.next;
n.prev = it;
n.next.prev = n;
n.prev.next = n;
```

Similarly, if it points to the node that we need to remove, we can remove that node with two operations:

```
it.prev.next = it.next;
it.next.prev = it.prev;
```

Circular Linked List

A circular linked list is created by simply linking the last element to the first element to create a cycle. A circular data structure can be incredibly useful because it is technically never ending unless it is empty. For processes that repeat indefinitely, a circular linked list may be the right way to ensure it continues to operate continuously.



Since we can structure a linked list in a circular manner, it raises the question of whether or not we can do something similar with an array. In fact, we can and this special configuration of an array is called a ring which we will briefly examine in the next chapter.

Iterating over the circular linked list only changes slightly with the big change being that we can no longer look for null to mark the end of the list. Instead, the tail end of the list is marked by pointing to the same element as the head pointer, so we can test for whether the head is next as a way to test for the tail of the list.

```
it = head;
while(it.next != head) {
    it = it.next;
}
```

We can doubly-link the circular linked list as well allowing us to traverse either way around the cycle. In a doubly-linked, circular linked-list, we can also access the tail in constant time via `head.prev`, allowing us to keep track of minimal extra references.

Review

1. What is the role of the head pointer in a linked-list?
2. Define the structure of a singly-linked linked-list node?
3. How is the end of a linked-list represented?
4. Analyze the algorithmic time complexity of append for a linked-list based List.
5. Analyze the algorithmic time complexity of remove for a linked-list based List.
6. Assuming the maintenance of a tail pointer, why is appending to the tail end of a singly-linked linked-list constant time complexity?
7. Assuming no tail pointer, why is appending to the tail end of a singly-linked linked-list linear time complexity?
8. Is it necessary to maintain both a head and a tail pointer for a linked-list to support all List interactions? Justify your answer.
9. Assess and demonstrate the feasibility of adding to the head of a singly-linked linked-list. If this operation is feasible, analyze the efficiency of adding to the head of a linked-list.
10. Assess and demonstrate the feasibility of removing from the head of a singly-linked linked-list. If this operation is feasible, analyze the efficiency of removing from the head of a linked-list.
11. Assess and demonstrate the feasibility of adding to the tail of a singly-linked linked-list. If this operation is feasible, analyze the efficiency of adding to the tail of a linked-list.
12. Assess and demonstrate the feasibility of removing from the tail of a singly-linked linked-list. If this operation is feasible, analyze the efficiency of removing from the tail of a linked-list.
13. Define the structure of a doubly-linked linked-list node
14. What improvements does a doubly-linked linked-list node bring to a List implementation? Justify your answer.
15. What additional costs does a doubly-linked linked-list node bring to a List implementation? Justify your answer.
16. Contrast the time complexity of list operations in a linked list against list operations in an array based list.
17. Contrast the space complexity of list operations in a linked list against list operations in an array based list.
18. Justify why a `while` loop is recommended for iteration in a linked list instead of a for loop?
19. What is an iterator?
20. How does using an iterator for iteration differ from indexed based iteration?
21. Why should iterator based iteration be used for linked lists over index based iteration?